

# 多场景或多领域RAG知识隔离架构

这是一个非常典型的多场景/多领域RAG知识隔离问题。直接将所有PDF混合进一个向量库确实会导致跨场景知识干扰（比如用户问“医疗报销流程”，模型却引用了“IT设备申请指南”），降低回答准确性和可信度。

以下是系统性的解决方案，结合架构设计、技术选型和具体操作步骤：

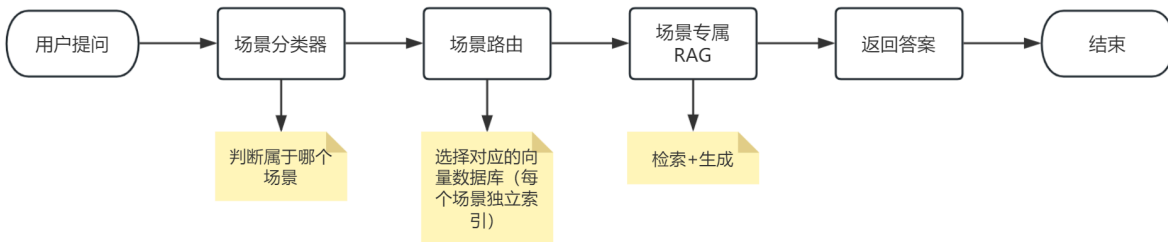
## 一、核心思路：场景路由 + 知识隔离

不要把所有知识混在一起，而是：

1. 先判断用户问题属于哪个场景
2. 只检索该场景对应的知识库
3. 用该场景知识生成答案

## 二、推荐架构：分层RAG + 动态路由

架构描述：



## 三、实现方案（3种主流方式）

### 方案1：多向量库 + 分类器路由（推荐）

适合场景数量固定（如20~100个），每个场景知识量中等。

步骤：

#### 1. 预处理阶段：

- 为每个PDF（场景）单独构建向量数据库（如 Chroma、Pinecone、Weaviate 的不同 collection/index）
- 例如：`hr_policy_db`, `it_support_db`, `finance_db`...

#### 2. 构建场景分类器：

- **方法A（轻量级）**：用规则+关键词（如包含“报销”→财务，“请假”→HR）
  - 不用大模型来实现，而是使用基于向量机的机器学习算法+TF-IDF 来实现
- **方法B（ML）**：训练一个文本分类模型（如BERT微调），输入问题，输出场景标签
- **方法C（LLM-based）**：用LLM做零样本分类（Prompt Engineering）

```
prompt = f"""
你是一个场景分类器。请判断以下问题属于哪个业务场景：
可选场景：[HR政策，IT支持，财务报销，行政管理，法务咨询]
问题：{user_query}
输出格式：仅返回场景名称，不要解释。
"""
```

### 3. 运行时流程：

- 用户提问 → 分类器输出场景标签（如“财务报销”）
- 系统自动选择 `finance_db` 进行检索
- 用检索结果 + 原始问题 → 生成答案

#### 优点：

- 知识完全隔离，无干扰
- 可单独更新某个场景知识库
- 检索效率高（只查一个小库）

#### 工具推荐：

- 向量库：**Chroma**（本地）、**Pinecone**（云）、**Qdrant**（高性能）
- 分类器：**scikit-learn**（SVM/TF-IDF）、**HuggingFace Transformers**（BERT）、**LLM API**

## 方案2：元数据过滤（Metadata Filtering）

适合使用支持元数据过滤的向量数据库（如 Pinecone, Weaviate, Qdrant）

#### 步骤：

##### 1. 构建统一向量库，但每条chunk添加场景标签：

```
# 示例：每个文本块附加 metadata
chunk = {
    "text": "报销需提供发票原件...",
    "metadata": {"scene": "finance", "source_pdf": "finance_policy_v2.pdf"}
}
```

##### 2. 检索时动态过滤：

- 先用分类器判断场景（同方案1）
- 检索时加上过滤条件：`filter={"scene": "finance"}`
- 向量库只返回该场景的chunks

#### 优点：

- 只需维护一个向量库，管理简单
- 支持跨场景扩展（新增场景只需加标签）

#### 注意：

- 需要向量数据库支持高效元数据过滤（Chroma免费版过滤性能较差）

## 方案3：HyDE + 查询重写（高级方案）

适合场景边界模糊、问题表述不明确的情况

结合查询理解和路由：

- 1. 用LLM将用户问题重写为更明确的查询（HyDE）
- 2. 用重写后的查询做场景分类 + 检索
- 3. 可进一步加入置信度阈值：如果分类器不确定，可返回“请明确您的问题属于哪个领域”

## 四、框架推荐

| 需求    | 推荐框架                             |
|-------|----------------------------------|
| 快速原型  | LlamaIndex（支持多索引 + 路由）           |
| 企业级部署 | LangChain + 自定义路由层               |
| 高性能检索 | Qdrant / Weaviate + FastAPI      |
| 低代码方案 | LlamaIndex Multi-Document Agents |

## 五、具体实现

使用 Python 实现一套 完整、可落地、开箱即用的多场景 RAG 解决方案，基于 LlamaIndex + Chroma（本地）或 Qdrant（高性能）+ 轻量级场景分类器，实现知识隔离、避免干扰。

✅ 最终目标

- 每个 PDF 对应一个业务场景（如 hr\_policy.pdf, it\_guide.pdf）
- 用户提问时，自动识别场景 → 只检索该场景知识库 → 生成精准答案
- 代码模块清晰，易于扩展新场景

### 1、技术栈选择

| 组件     | 推荐方案                        | 理由                     |
|--------|-----------------------------|------------------------|
| 向量数据库  | Chroma（本地开发）或 Qdrant（生产）    | 支持多 collection / 元数据过滤 |
| RAG 框架 | LlamaIndex                  | 原生支持多索引、路由、PDF 加载      |
| 场景分类器  | 规则 + LLM 零样本分类              | 无需训练，快速上线              |
| LLM 后端 | OpenAI / Ollama / DashScope | 可替换                    |

## 2、项目结构

```
multi_scene_rag/
├─ data/
│   └─ hr/                # HR场景PDF
│       └─ 中华人民共和国劳动法.pdf
│   └─ it/                # IT场景PDF
│       └─ IT运维服务规范.pdf
│   └─ finance/          # 财务场景PDF
│       └─ 公司日常报销流程.pdf
├─ storage/              # 向量库存储（自动生成）
├─ config.py             # 场景配置
├─ classifier.py         # 场景分类器
├─ rag_engine.py         # RAG核心逻辑
└─ app.py                # 主程序入口
```

## 3、代码实现

### 3.1. 安装依赖

创建一个 conda 环境

```
llama-index
llama-index-readers-file
llama-index-vector-stores-chroma
llama-index-llms-dashscope
llama-index-embeddings-dashscope
```

### 3.2. 定义场景

```
# config.py
SCENES = {
    "hr": {
        "name": "人力资源政策",
        "keywords": ["请假", "年假", "入职", "离职", "合同", "社保"],
        "path": "./data/hr"
    },
    "it": {
        "name": "IT支持",
        "keywords": ["电脑", "网络", "账号", "打印机", "软件", "VPN"],
        "path": "./data/it"
    },
    "finance": {
        "name": "财务报销",
        "keywords": ["报销", "发票", "差旅", "付款", "预算", "费用"],
        "path": "./data/finance"
    }
}
```

### 3.3. 场景分类器（规则 + LLM）

```
# classifier.py
import os
from openai import OpenAI
from config import SCENES

client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

def classify_scene_by_keywords(query: str) -> str | None:
    """基于关键词快速匹配"""
    for scene, info in SCENES.items():
        if any(kw in query for kw in info["keywords"]):
            return scene
    return None

def classify_scene_by_llm(query: str) -> str:
    """LLM零样本分类（兜底）"""
    scene_names = list(SCENES.keys())
    scene_descs = [f"{k}: {v['name']}" for k, v in SCENES.items()]

    prompt = f"""
你是一个企业知识库路由系统。请根据用户问题，判断其最可能属于以下哪个业务场景：
{chr(10).join(scene_descs)}

问题: {query}
要求: 只输出场景英文标识（如 hr, it, finance），不要解释。
"""

    try:
        response = client.chat.completions.create(
            model="qwen-plus",
            messages=[{"role": "user", "content": prompt}],
            temperature=0.0,
            max_tokens=10
        )
        pred = response.choices[0].message.content.strip().lower()
        return pred if pred in SCENES else "hr" # 默认回退到hr
    except Exception as e:
        print(f"LLM分类失败: {e}")
        return "hr"

def classify_scene(query: str) -> str:
    """主分类函数：先关键词，再LLM"""
    scene = classify_scene_by_keywords(query)
    if scene:
        return scene
    return classify_scene_by_llm(query)
```

### 3.4. 多场景RAG引擎

```
# rag_engine.py
import os
from llama_index.core import (
    Settings,
    VectorStoreIndex,
    SimpleDirectoryReader,
    StorageContext,
    load_index_from_storage
)
from llama_index.vector_stores.chroma import ChromaVectorStore
import chromadb
from llama_index.llms.dashscope import DashScope, DashScopeGenerationModels
from llama_index.embeddings.dashscope import (
    DashScopeEmbedding,
    DashScopeTextEmbeddingModels,
    DashScopeTextEmbeddingType,
)
from config import SCENES
from classifier import classify_scene

# 设置全局 LLM 和 Embedding Model
Settings.llm = DashScope(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    model_name=DashScopeGenerationModels.QWEN_MAX
)
Settings.embed_model = DashScopeEmbedding(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    model_name=DashScopeTextEmbeddingModels.TEXT_EMBEDDING_V1,
    text_type=DashScopeTextEmbeddingType.TEXT_TYPE_DOCUMENT
)

class MultiSceneRAG:
    def __init__(self):
        self.indices = {}
        self._init_indices()

    def _init_indices(self):
        """为每个场景构建或加载向量索引"""
        client = chromadb.PersistentClient(path="./storage/chroma_db")

        for scene, info in SCENES.items():
            print(f"Loading index for scene: {scene}")
            collection = client.get_or_create_collection(f"scene_{scene}")
            vector_store = ChromaVectorStore(chroma_collection=collection)

            storage_context = StorageContext.from_defaults(
                vector_store=vector_store,
            )

            persist_dir = f"./storage/{scene}"

            # 检查完整的持久化目录是否存在（包含docstore.json等文件）
            docstore_path = os.path.join(persist_dir, "docstore.json")
```

```

        if os.path.exists(f"./storage/{scene}") and
os.path.exists(docstore_path):
            # 加载已有索引
            try:
                storage_context = StorageContext.from_defaults(
                    vector_store=vector_store,
                    persist_dir=persist_dir
                )
                index = load_index_from_storage(storage_context)
                print(f"✅ Successfully loaded existing index for scene:
{scene}")

            except Exception as e:
                print(f"⚠️ Failed to load existing index for scene {scene}:
{e}")

                print(f"🔄 Rebuilding index for scene: {scene}")
                # 如果加载失败, 重新构建索引
                index = self._build_new_index(info["path"], storage_context,
persist_dir)

            else:
                print(f"🔄 Building new index for scene: {scene}")
                # 首次构建索引或目录不存在
                index = self._build_new_index(info["path"], storage_context,
persist_dir)

        self.indices[scene] = index.as_query_engine()

def _build_new_index(self, data_path, storage_context, persist_dir):
    """构建新的索引"""
    if not os.path.exists(data_path):
        raise FileNotFoundError(f>Data path does not exist: {data_path}")

    documents = SimpleDirectoryReader(data_path).load_data()
    index = VectorStoreIndex.from_documents(
        documents,
        storage_context=storage_context
    )

    # 确保持久化目录存在
    os.makedirs(persist_dir, exist_ok=True)
    index.storage_context.persist(persist_dir=persist_dir)
    print(f"✅ Successfully built and persisted index for scene")

    return index

def query(self, user_query: str) -> str:
    scene = classify_scene(user_query)
    print(f"🔍 路由到场景: {SCENES[scene]['name']} ({scene})")

    query_engine = self.indices[scene]
    response = query_engine.query(user_query)
    return str(response)

```

### 3.5. 主程序

```
# app.py
from rag_engine import MultiSceneRAG

if __name__ == "__main__":
    rag = MultiSceneRAG()

    while True:
        query = input("\n请输入您的问题（输入 'quit' 退出）：")
        if query.lower() == 'quit':
            break
        answer = rag.query(query)
        print(f"✅ 答案:\n{answer}\n")
```

## 4、测试

1. **准备数据**：将每个场景的 PDF 放入 `data/{scene}/` 目录
2. **首次运行**： `python app.py` → 自动构建所有场景的向量索引（存储在 `./storage/`）
3. **提问测试**：
  - “用人单位在哪些节日期间应当依法安排劳动者休假？” → 路由到 `hr`
  - “公司日常报销流程及步骤？” → 路由到 `finance`
  - “IT运维服务规范的参考标准？” → 路由到 `it`

## 5、为什么这个方案有效

| 问题   | 本方案如何解决                                 |
|------|-----------------------------------------|
| 知识干扰 | 每个场景独立向量库，物理隔离                          |
| 路由不准 | 关键词 + LLM 双保险                           |
| 扩展困难 | 新增场景只需加 PDF + 更新 <code>config.py</code> |
| 性能差  | 只检索小库，速度快                               |

## 6、建议

1. 先用 Chroma 本地跑通 demo
2. 测试 3~5 个真实问题，验证路由准确性
3. 根据效果决定是否引入更复杂的分类器（如微调 BERT）



## 六、总结

| 方案         | 适用场景       | 复杂度   | 推荐指数 |
|------------|------------|-------|------|
| 多向量库 + 分类器 | 场景清晰、数量适中  | ★★☆   | ★★★★ |
| 元数据过滤      | 场景较多、需统一管理 | ★★★   | ★★★  |
| LLM动态路由    | 场景模糊、需高精度  | ★★★★★ | ★★★  |

**首选建议：**  
👉 用 LlamaIndex 或 LangChain 构建多索引系统 + 轻量级场景分类器（规则+LLM），实现知识隔离与精准检索。